Minimization Algorithms: From Newton–Raphson to Adam

Yuval Lavie

August 10, 2026

A lineage of update rules for $\min_\theta f(\theta)$, each fixing a weakness of its predecessor. All are implemented from scratch and raced on the Rosenbrock valley in `optimizers.py`. Throughout, $g_k = \nabla f(\theta_k)$ and η is the step size (learning rate).

1 Second order: Newton–Raphson

Newton's method finds roots of an equation; *optimization is root-finding on the gradient* ($\nabla f = 0$). Modeling f locally by its second-order Taylor expansion and jumping to that model's minimum:

$$\theta_{k+1} \;=\; \theta_k - H_k^{-1} g_k, \qquad H_k = \nabla^2 f(\theta_k). \tag{1}$$

Near a minimum with $H \succ 0$, convergence is *quadratic* — the number of correct digits doubles per step. The costs: computing and solving a $d \times d$ Hessian per step ($O(d^3)$ — hopeless for large models), and far from the optimum H_k may fail to be positive definite, sending (1) uphill (the script damps it when that happens). Everything after this section is an attempt to keep some of Newton's geometry while paying first-order prices.

2 First order: gradient descent

$$\theta_{k+1} \;=\; \theta_k - \eta\, g_k. \tag{2}$$

For L-smooth convex f and $\eta \le 1/L$: $f(\theta_k) - f^* = O(1/k)$; add μ-strong convexity and the rate is linear (geometric) with contraction governed by the condition number $\kappa = L/\mu$. That κ-dependence is GD's weakness: on an ill-conditioned valley (Rosenbrock) the largest stable η is set by the steep direction, leaving the shallow direction to crawl.

SGD. Replace g_k by an unbiased estimate from one sample or a mini-batch (as in `bernoulli/`, `gaussian/`, `linear-regression/`): cost per step drops from $O(n)$ to $O(1)$ samples; a constant step leaves a noise ball, handled by averaging (Polyak–Ruppert) or decaying steps. All methods below take stochastic gradients in practice.

3 Momentum: heavy ball and Nesterov

Heavy ball (Polyak). Accumulate a velocity:

$$v_{k+1} = \beta v_k + g_k, \qquad \theta_{k+1} = \theta_k - \eta\, v_{k+1}, \qquad \beta \in [0, 1). \tag{3}$$

Along a persistent downhill direction the geometric series boosts the effective step by up to $1/(1-\beta)$; across the valley, alternating gradients cancel in v. Physically: a ball with inertia rolling on the loss surface with friction $1-\beta$.

Nesterov accelerated gradient. Evaluate the gradient at the *look-ahead* point:

$$v_{k+1} = \beta v_k + \nabla f\big(\theta_k - \eta\beta v_k\big), \qquad \theta_{k+1} = \theta_k - \eta\, v_{k+1}. \tag{4}$$

The correction "looks where the momentum is taking you before deciding." On smooth convex problems it achieves $f(\theta_k) - f^* = O(1/k^2)$ — provably optimal for first-order methods, versus GD's $O(1/k)$.

4 Quasi-Newton: BFGS and L-BFGS

Estimate curvature from gradient differences instead of computing H. With $s_k = \theta_{k+1} - \theta_k$ and $y_k = g_{k+1} - g_k$, BFGS maintains an inverse-Hessian estimate B_k satisfying the *secant condition* $B_{k+1}y_k = s_k$:

$$B_{k+1} = \big(I - \rho_k s_k y_k^\top\big) B_k \big(I - \rho_k y_k s_k^\top\big) + \rho_k s_k s_k^\top, \qquad \rho_k = \frac{1}{y_k^\top s_k}, \tag{5}$$

stepping $\theta_{k+1} = \theta_k - \alpha_k B_k g_k$ with a line search ($y_k^\top s_k > 0$ keeps $B \succ 0$). Superlinear convergence at first-order cost per step. *L-BFGS* stores only the last m pairs (s, y) and applies B_k implicitly — $O(md)$ memory, the standard for smooth medium-scale problems. (Quasi-Newton dislikes stochastic gradients: noisy y_k corrupts the curvature estimate, one reason deep learning went the adaptive route instead.)

5 Adaptive per-coordinate steps: AdaGrad, RMSProp, Adam

One global η misfits problems whose coordinates have very different scales. Give each coordinate its own step from its gradient history (all operations below are element-wise).

AdaGrad:

$$s_{k+1} = s_k + g_k^2, \qquad \theta_{k+1} = \theta_k - \eta\, \frac{g_k}{\sqrt{s_{k+1}} + \varepsilon}. \tag{6}$$

Frequently/steeply updated coordinates get shrinking steps. Flaw: s only grows, so all steps decay to zero eventually.

RMSProp forgets exponentially, fixing the decay:

$$s_{k+1} = \rho\, s_k + (1-\rho)\, g_k^2, \qquad \theta_{k+1} = \theta_k - \eta\, \frac{g_k}{\sqrt{s_{k+1}} + \varepsilon}. \tag{7}$$

Adam = RMSProp + momentum + bias correction:

$$\begin{aligned} m_{k+1} &= \beta_1 m_k + (1-\beta_1)\, g_k, \quad \hat{m} = m_{k+1}/(1-\beta_1^{k+1}), \\ v_{k+1} &= \beta_2 v_k + (1-\beta_2)\, g_k^2, \quad \hat{v} = v_{k+1}/(1-\beta_2^{k+1}), \end{aligned} \qquad \theta_{k+1} = \theta_k - \eta\, \frac{\hat{m}}{\sqrt{\hat{v}} + \varepsilon}. \tag{8}$$

The $(1-\beta^k)$ corrections debias the zero-initialized moment estimates in early iterations. Defaults $\beta_1 = 0.9$, $\beta_2 = 0.999$ have proven remarkably robust — Adam is the de-facto default of deep learning. *AdamW* decouples weight decay from the adaptive gradient step ($\theta \leftarrow \theta - \eta\lambda\theta$ applied separately), the variant modern training recipes actually use.

6 What the race shows

On Rosenbrock from $(-1.5, 2)$ (see the script's log-scale figure): plain GD crawls along the valley floor; momentum and Nesterov accelerate it substantially; the adaptive family navigates the bad scaling without hand-tuned per-direction steps; and the curvature methods — Newton and BFGS — drop to machine precision in tens of steps, which is what quadratic/superlinear convergence looks like on a plot. No single winner: the ranking would change with dimension, stochasticity, and cost per step — which is precisely why the zoo exists.